

DocSearch

Dokumentasi Aplikasi: Portal OCR, Library Cloud,
Pencarian Dokumen, dan Ask AI

Dokumen ini menjelaskan DocSearch secara menyeluruh: konteks produk, Paperless-ngx, arsitektur, database, alur teknis (flowchart), modul per halaman, keamanan multi-user, model OpenAI & biaya, serta operasional dan FAQ.

Bahasa: Indonesia · Format: dokumentasi teknis (A4) untuk pembaca non-dev & teknis

Versi dokumen: Agustus 2026

Output: DocSearch-Dokumentasi-Aplikasi.pdf

Daftar isi

A. Ringkasan & konteks

1. Ringkasan aplikasi
2. Masalah yang diselesaikan

B. Paperless-ngx mendalam

3. Paperless-ngx: asal, fungsi, kelebihan, batasan, dan peran di DocSearch

C. Arsitektur & stack

4. Arsitektur DocSearch
5. Stack teknologi

D. Database lengkap

6. Database & model data

E. Alur teknis mendalam

7. Alur teknis (SyncFile, retry, auto-scan, embedding, ingest, Ask AI, Search)

F. Modul per halaman

8. Modul portal: Ask AI, Search, Library, Admin, Profil

G. Keamanan & multi-user

9. Keamanan, isolasi data, role, audit, rate limit

H. OpenAI models & biaya

10. Model OpenAI, kapan dipanggil, dan audit biaya

I. Operasional

11. Deploy, backup, health, dan tuning 8 GB RAM

J. Lampiran

12. Glosarium, FAQ, checklist go-live

A. Ringkasan & konteks

1. Ringkasan aplikasi

DocSearch adalah portal web untuk organisasi yang menyimpan banyak PDF di cloud (contoh: Nextcloud/ownCloud lewat WebDAV). Portal ini membantu pengguna **mengambil dokumen dari cloud, mengubahnya menjadi teks lewat OCR, mencari isi dokumen, dan bertanya ke AI** berdasarkan dokumen yang sudah diproses.

Intinya: cloud tetap jadi sumber file asli; DocSearch jadi jembatan pintar antara file PDF, mesin OCR (Paperless-ngx), database aplikasi, antrean pekerjaan, dan layanan AI (OpenAI).

Komponen utama dalam satu kalimat

Komponen	Peran singkat
Portal web (Next.js)	UI login, Library, Search, Ask AI, Admin, Profil
sync-worker	Unduh file dari cloud, antre OCR, reconcile, embedding backfill
Paperless-ngx	Mesin OCR & penyimpanan dokumen terproses
PostgreSQL	Database aplikasi (app) + database Paperless (paperless)
Redis	Antre pekerjaan BullMQ & broker Paperless
OpenAI (opsional)	Embedding RAG + jawaban Ask AI / ringkasan search

Gambaran besar (satu baris)



Gambar 1. DocSearch sebagai jembatan cloud → OCR → pencarian & AI.

2. Masalah yang diselesaikan

Di banyak organisasi, dokumen sudah digital (PDF di cloud), tapi belum siap dipakai sebagai pengetahuan yang bisa dicari dan ditanyakan:

- **Sulit dicari isinya:** nama file sering tidak mewakili isi. Mencari berarti buka folder dan PDF satu per satu.
- **OCR tidak terpusat:** scan/OCR manual tidak skala untuk ratusan atau ribuan file.
- **Status tidak jelas:** mana yang sudah dibaca mesin, mana yang gagal, mana yang kosong (0 byte).
- **Belum siap untuk tanya AI:** AI butuh teks hasil OCR + indeks; tanpa pipeline, Ask AI tidak andal.
- **Isolasi per orang:** tiap pegawai punya kredensial cloud sendiri; sistem harus menjaga agar dokumen tidak tercampur.

Alur sebelum DocSearch



Gambar 2. Alur kerja lama: banyak langkah manual.

Alur setelah DocSearch



Gambar 3. Alur baru: mesin membantu mencari dan menjawab.

B. Paperless-ngx mendalam

3. Paperless-ngx: asal, fungsi, kelebihan, batasan, dan peran di DocSearch

3.1 Apa itu Paperless-ngx?

Paperless-ngx adalah perangkat lunak **open source** untuk mengelola dokumen digital dengan OCR (Optical Character Recognition). Proyek ini adalah kelanjutan komunitas dari proyek sebelumnya yang dikenal sebagai **Paperless** / Paperless-ng. Tujuannya: mengubah tumpukan PDF/scan menjadi arsip yang bisa dicari teksnya, diberi tag, dan dikelola lewat antarmuka web serta API.

Nama “paperless” merujuk ide kantor tanpa kertas: dokumen fisik di-scan, lalu hidup sebagai file digital yang tetap bisa dicari seperti teks biasa.

3.2 Asal usul singkat

- Awalnya komunitas membangun sistem arsip dokumen pribadi/rumah tangga (invoice, surat, dll.).
- Seiring waktu berkembang jadi **Paperless-ngx**: lebih modern, Docker-friendly, aktif dikembangkan.
- Biasanya dijalankan self-hosted (di server sendiri), bukan SaaS berbayar wajib.
- Mesin OCR di balik layar umumnya memakai **OCRmyPDF / Tesseract** (tergantung konfigurasi image).

3.3 Fungsi umum

Fungsi	Penjelasan singkat
Consume / ingest	Memantau folder; file baru otomatis diproses
OCR	Mengenali teks di PDF (termasuk scan gambar)
Full-text search	Mencari kata di dalam isi dokumen (di UI Paperless)
Metadata & tag	Judul, tanggal, correspondent, tags, dll.
Preview & unduh	Melihat/mengunduh PDF hasil kelola
API REST + token	Integrasi dari aplikasi lain (DocSearch memakai ini)
Multi-bahasa OCR	Bisa set bahasa, mis. English + Indonesia

3.4 Kelebihan

- **Open source & self-hosted**: data tetap di infrastruktur organisasi.
- **OCR matang**: sudah ada pipeline consume → OCR → indeks, tidak perlu bangun dari nol.
- **API lengkap**: cocok digabung dengan portal custom seperti DocSearch.
- **Docker**: relatif mudah di-deploy bersama PostgreSQL & Redis.
- **Bahasa OCR bisa dikonfigurasi**: di DocSearch di-set untuk Inggris + Indonesia (eng+ind).
- **Ekosistem mapan**: dokumentasi komunitas, image resmi, praktik operasional sudah banyak.

3.5 Batasan / kritik

Catatan kritis: Paperless hebat sebagai mesin OCR & arsip, tetapi **bukan portal multi-tenant cloud browser**. Tanpa lapisan aplikasi tambahan, Paperless kurang ideal untuk skenario “tiap pegawai login, browse WebDAV sendiri, lalu Ask AI dari dokumen miliknya”.

- UI Paperless lebih ke arsip dokumen, bukan Library cloud seperti Explorer.
- Isolasi per-user cloud credentials bukan fokus utama Paperless; itu tanggung jawab portal DocSearch.
- OCR berat di CPU/RAM; di server kecil harus dibatasi worker agar tidak OOM.

- Kualitas OCR bergantung kualitas PDF (scan buram / file 0 byte tetap bermasalah).
- Fitur AI chat / embedding semantik bukan inti Paperless; DocSearch menambahnya lewat OpenAI.

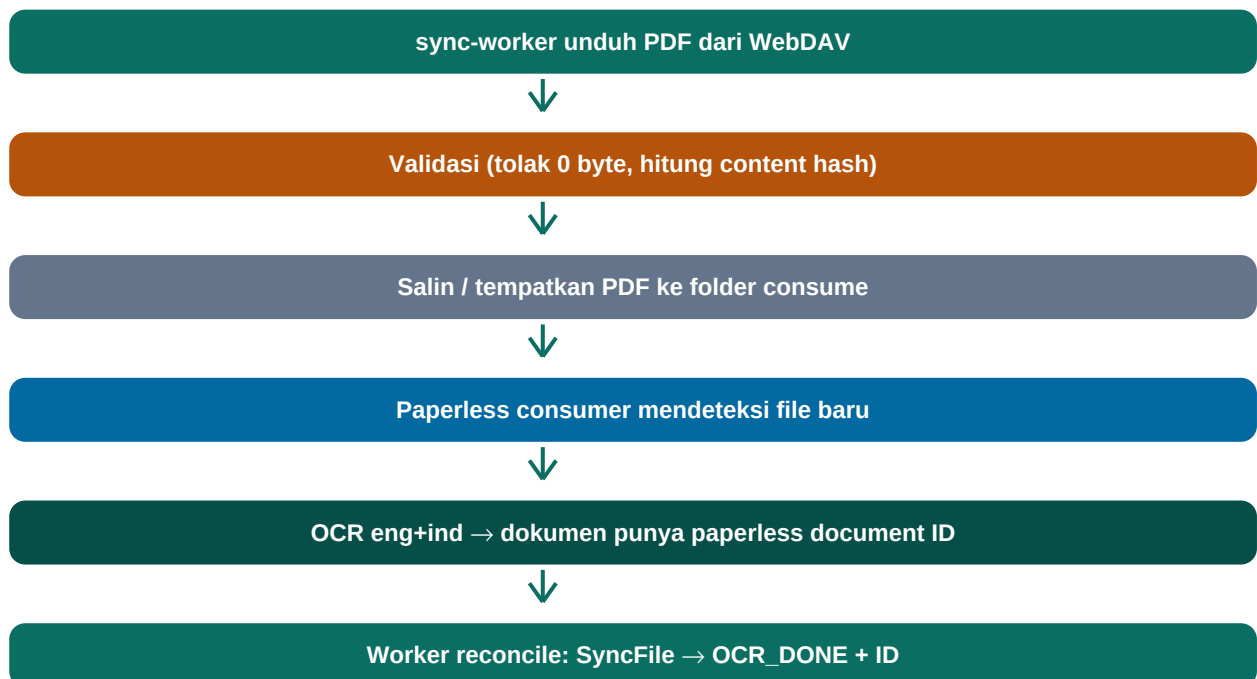
3.6 Apa yang dipakai Paperless di DocSearch?

Di DocSearch, Paperless-ngx dipakai sebagai **mesin OCR + penyimpanan dokumen terproses**, bukan sebagai satu-satunya UI untuk pengguna akhir. Pengguna sehari-hari memakai portal DocSearch.

Aspek	Konfigurasi / pemakaian di DocSearch
Image	ghcr.io/paperless-ngx/paperless-ngx (Docker Compose)
Database	PostgreSQL, database terpisah bernama paperless
Broker / Redis	Redis untuk antrean tugas internal Paperless
OCR language	eng+ind (Inggris + Indonesia)
Consume folder	Volume bersama yang diisi sync-worker (PDF masuk sini)
API token	PAPERLESS_API_TOKEN / PAPERLESS_TOKEN untuk REST API
Akses UI Paperless	Lokal/internal (bukan pintu utama user portal)
Yang dipakai portal	ID dokumen, preview, unduh, meta/teks untuk chunk

3.7 Apa yang terjadi di folder consume?

Folder consume adalah “kotak masuk” Paperless. Sync-worker menaruh PDF yang sudah diunduh dari cloud ke folder ini. Paperless memantau folder (polling), lalu memproses file: OCR, ekstraksi teks, penyimpanan media, dan pembuatan entri dokumen dengan ID unik.



Gambar 4. Siklus hidup file di folder consume.

3.8 Checksum / Paperless document ID vs SyncFile

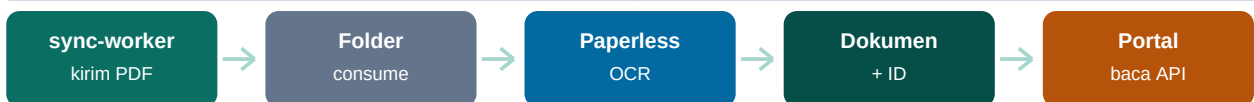
Setiap SyncFile di database app menyimpan jejak file cloud milik satu user. Dua field kunci menghubungkan dunia app dengan Paperless:

Field SyncFile	Arti
content_hash	Checksum isi file (setelah unduh). Dipakai deteksi duplikat & pencocokan ke Paperless.
paperless_document_id	ID dokumen di Paperless setelah OCR sukses (atau diwarisi dari duplikat).
remote_path	Path unik file di cloud untuk user tersebut (unik bersama userId).
sync_status	Posisi file di pipeline (DISCOVERED ... OCR_DONE / FAILED / SKIPPED).

Reconcile: worker mencari dokumen Paperless berdasarkan checksum. Jika ketemu, SyncFile di-update ke OCR_DONE dan diisi paperless_document_id. Jika hash sama dengan file lain yang sudah punya ID, file bisa di-SKIPPED (duplikat isi) tanpa OCR ulang.

3.9 API Paperless yang dipakai portal

Dipakai portal / worker	Tidak jadi UI utama
Preview PDF dokumen terproses	UI arsip / tag / correspondent Paperless untuk end-user
Download / raw dokumen via ID	Full-text search UI bawaan Paperless sebagai muka utama
Meta dokumen & pencarian by checksum (reconcile)	Manajemen user Paperless untuk pegawai organisasi
Token API untuk worker & app	Ingest manual lewat UI Paperless (biasanya diganti consume dari worker)



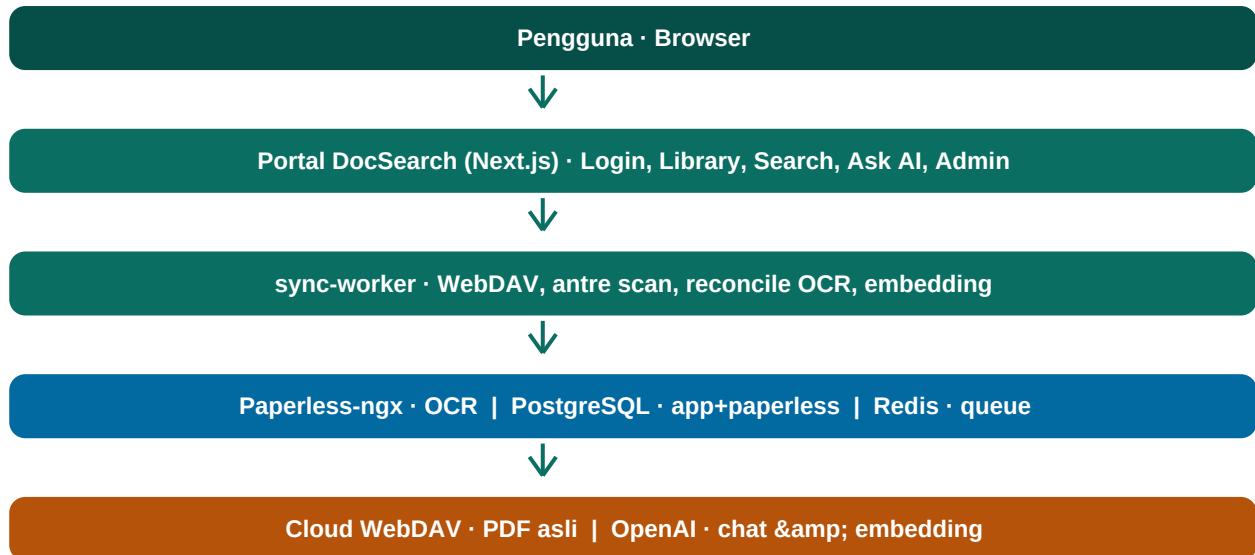
Gambar 5. Paperless sebagai mesin tengah; portal adalah muka pengguna.

C. Arsitektur & stack

4. Arsitektur DocSearch

DocSearch terdiri dari beberapa layanan yang saling berhubungan. Pengguna hanya melihat portal web. Di belakang ada worker, OCR, dua database logis, antrian Redis, sumber cloud, dan (opsional) OpenAI.

Lapisan sistem



Gambar 6. Lapisan arsitektur dari pengguna sampai sumber data & AI.

Pembagian tanggung jawab

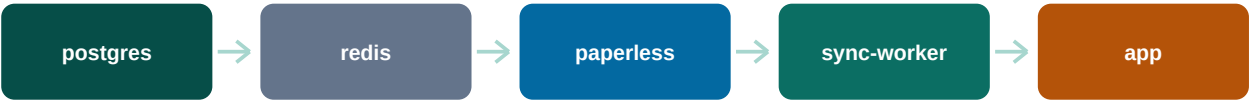
Layanan	Tanggung jawab	Tidak mengerjakan
Portal app	UI, auth, API, search/chat ke user	OCR berat di request HTTP
sync-worker	Ambil cloud, antre OCR, update status, embed	UI pengguna
Paperless	OCR & simpan dokumen terproses	Browse WebDAV per-user
PostgreSQL (app)	User, SyncFile, chunk, audit, chat	File biner PDF besar
Redis	Job queue BullMQ + broker Paperless	Sumber kebenaran bisnis jangka panjang

5. Stack teknologi

Stack dipilih agar self-hosted, bisa di-Docker-kan, dan cukup ringan untuk server menengah (rekomendasi praktis: sekitar 8 GB RAM).

Lapisan	Teknologi	Keterangan
Frontend + API	Next.js 16, React 19, TypeScript	App Router, API routes
Auth	NextAuth (Auth.js) v5	Session login, role ADMIN/USER
ORM	Prisma 6	Schema & migrasi PostgreSQL (DB app)
Styling	Tailwind CSS 4	UI portal
Worker	Node.js + TypeScript	Proses background sync
Queue	BullMQ + Redis	Job scan / ingest
Cloud client	webdav (npm)	List / download file cloud
OCR	Paperless-ngx	Consume + OCR eng+ind
Database	PostgreSQL 16	DB app + DB paperless
Cache/broker	Redis 7	Queue Paperless & BullMQ
AI chat	OpenAI OPENAI_MODEL	Default gpt-4o-mini
AI embedding	OPENAI_EMBEDDING_MODEL	Default text-embedding-3-small
Deploy	Docker Compose + scripts	server-up / deploy-remote

Mengapa bukan “semua di Paperless saja”? Karena kebutuhan bisnis adalah browse cloud per-user, status sync, Ask AI, audit biaya AI, dan UX portal. Paperless di sini adalah fondasi OCR yang kuat; portal + worker menutup celah produk.



Gambar 7. Lima layanan inti Compose. App adalah pintu pengguna.

D. Database lengkap

6. Database & model data

Secara fisik ada **satu server PostgreSQL**, tetapi secara logis ada dua database penting:

- **Database paperless**: dipakai Paperless-ngx untuk dokumen OCR, metadata internal, media references, dsb.
- **Database app**: dipakai portal & sync-worker untuk user, status sync, chat, audit, embedding chunk.

Pemisahan ini menjaga Paperless tetap “standar”, sementara logika bisnis DocSearch tidak mengotori skema internal Paperless.



Gambar 8. Satu Postgres, dua database logis.

6.1 ER sederhana (relasi)

Dari	Ke	Relasi / catatan
users	sync_files	1 : N (setiap file milik satu user)
users	scan_jobs	1 : N (job dipicu user / null jika sistem)
users	document_chunks	1 : N (chunk terisolasi per user)
users	chat_conversations	1 : N
chat_conversations	chat_messages	1 : N
users	cloud_favorites	1 : N (path unik per user)
users	audit_logs	1 : N (user boleh null untuk aksi sistem)
sync_files	document_chunks	1 : N (teks+embedding per file)
app_settings	-	Key-value global (auto-scan, dll.)
sync_files.paperless_document_id	Paperless (DB paperless)	Referensi silang logis, bukan FK SQL

6.2 Field penting per tabel

Tabel	Field / isi penting
users	email, name, role (ADMIN/USER), password_hash (bcrypt), bappenas_url, encrypted_bappenas_username/password, last_sync_at
sync_files	remote_path, file_name, file_size, content_hash, sync_status, paperless_document_id, error_message, ocr_pending_at
document_chunks	sync_file_id, paperless_document_id, chunk_index, content, embedding (JSON float[]), content_hash, token_estimate

Tabel	Field / isi penting
scan_jobs	job_type (full_scan / ingest_selected), status, phase, total/processed/skipped/failed/new, selected_paths, current_file
audit_logs	action, meta (JSON string: detail search/chat/admin/biaya), created_at
chat_conversations	title, scope (all / docs / folder), updated_at
chat_messages	role (USER/ASSISTANT), content, citations (JSON sumber)
cloud_favorites	path, last_opened_at, last_synced_at (delta auto-scan)
app_settings	key/value: auto_scan_enabled, auto_scan_interval_minutes, last_run_at

6.3 Status SyncFile detail + arti UI

Setiap file yang dilacak punya status. UI Library menampilkan progress berdasarkan status ini.



Gambar 9. Jalur sukses status SyncFile (ringkas).

Status	Arti teknis	Arti di UI (kira-kira)
DISCOVERED	Terdaftar / ditemukan, belum diunduh	Menunggu unduh
DOWNLOADING	Sedang diunduh dari WebDAV	Mengunduh...
QUEUED	Siap / menunggu masuk consume OCR	Antre OCR
OCR_PENDING	Sudah di consume; Paperless memproses	OCR berjalan...
OCR_DONE	Reconcile sukses; siap Search/Ask AI	Siap dipakai
SKIPPED	Duplikat / sudah ada (hash sama + ID)	Sudah ada (lewati)
FAILED	Gagal (0B, timeout, unduh, dll.)	Gagal · bisa retry

Kasus khusus file 0 byte: file kosong tidak dikirim ke OCR. Sistem menandainya FAILED dengan pesan jelas agar antrean tidak “nyangkut”.

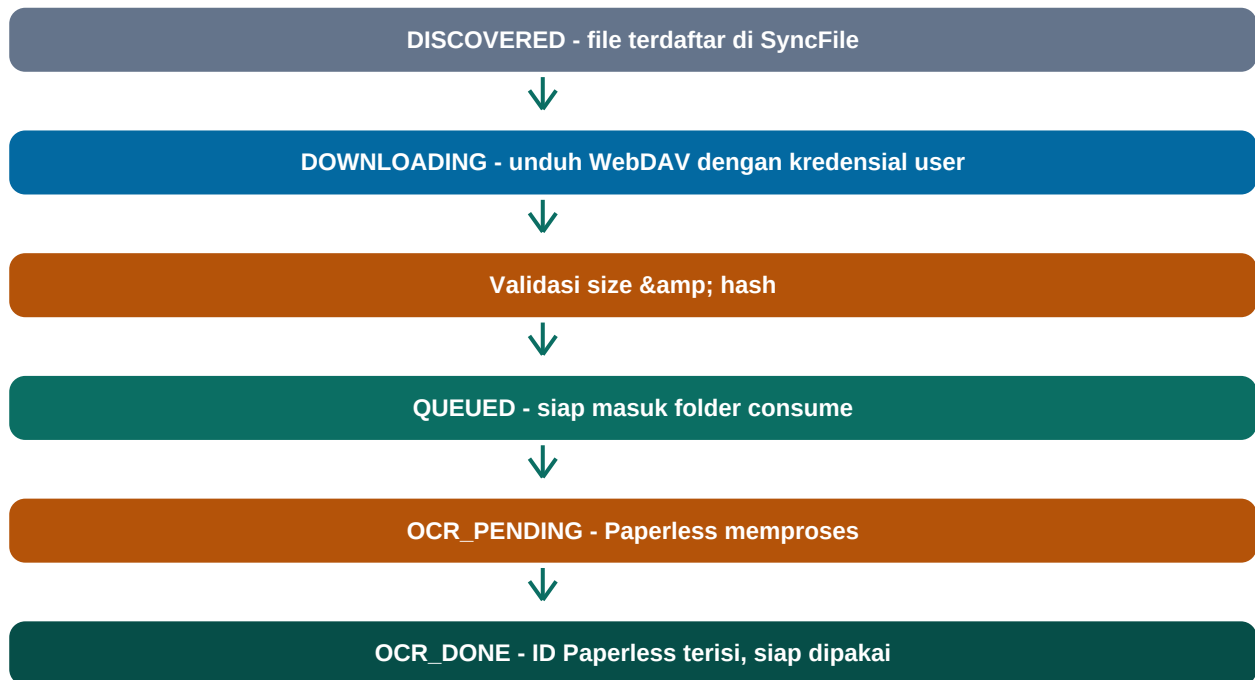
E. Alur teknis mendalam

7. Alur teknis (SyncFile, retry, auto-scan, embedding, ingest, Ask AI, Search)

Bab ini merinci pipeline yang sering ditanyakan saat operasi dan troubleshooting. Banyak diagram vertikal dan horizontal agar alur mudah diikuti.

7.1 Status SyncFile: pipeline sukses + cabang gagal

Jalur bahagia (happy path) berakhir di OCR_DONE. Cabang lain: SKIPPED (duplikat), FAILED (0 byte, timeout OCR, error unduh), atau tetap OCR_PENDING sampai timeout.



Gambar 10. Pipeline sukses SyncFile.

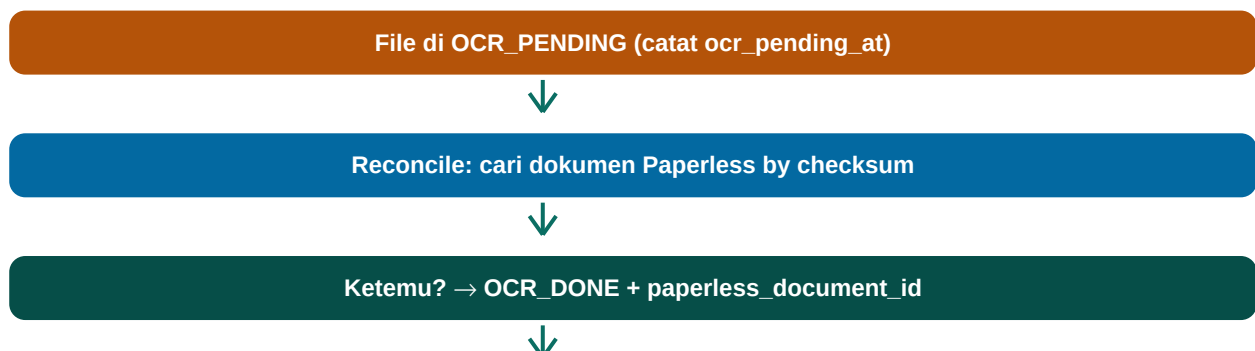
Cabang dari validasi / reconcile

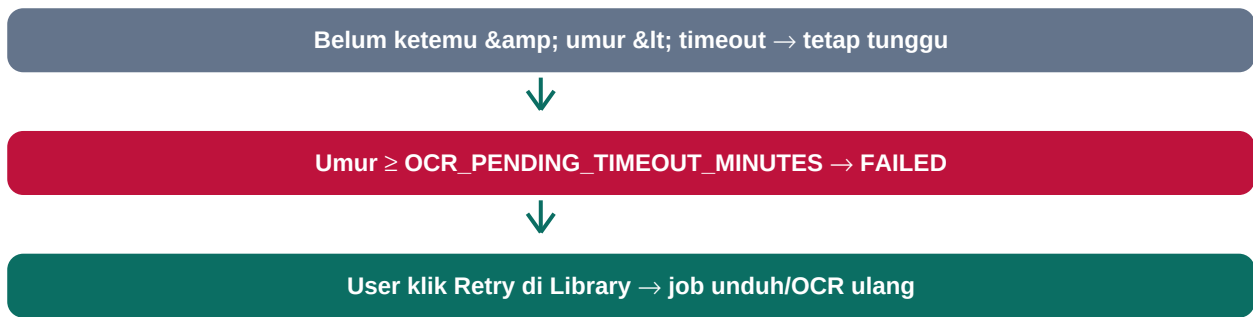


Gambar 11. Cabang SKIPPED / FAILED (0B / timeout) di samping jalur utama.

7.2 Retry, gagal, file 0 byte, timeout OCR

Worker memantau file yang lama di OCR_PENDING. Default timeout diatur oleh **OCR_PENDING_TIMEOUT_MINUTES** (default **180** menit). Setelah lewat, status diubah ke FAILED dengan pesan agar user bisa retry dari Library.



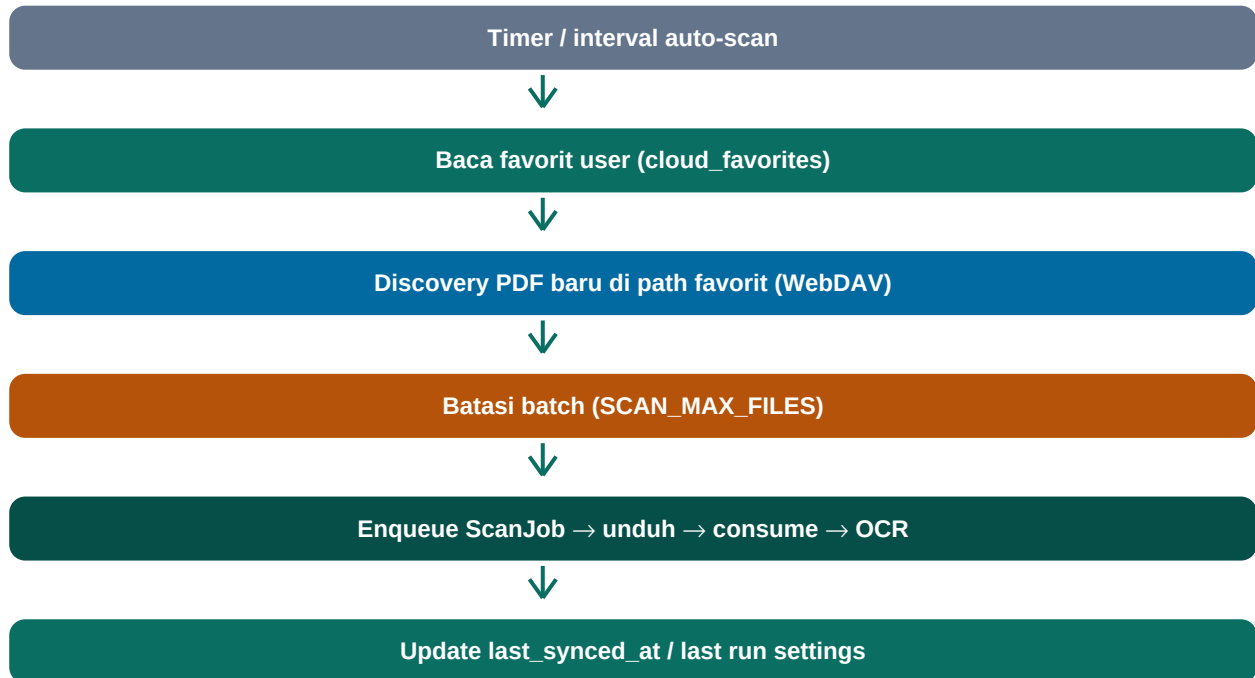


Gambar 12. Reconcile, timeout OCR, dan retry.

File 0 byte: tidak dikirim ke consume. Langsung FAILED dengan pesan bahwa file kosong tidak bisa di-OCR. Perbaiki/ganti file di cloud, lalu retry.

7.3 Auto-scan: favorit → discovery → batch

Auto-scan menjalankan ingest latar belakang untuk folder favorit user. Diaktifkan lewat Admin (app_settings) dan/atau dipaksa oleh env **AUTO_SCAN_ENABLED=true**. Interval default biasanya 60 menit (auto_scan_interval_minutes). Jumlah file per batch dibatasi **SCAN_MAX_FILES** (default di Compose sering 20).

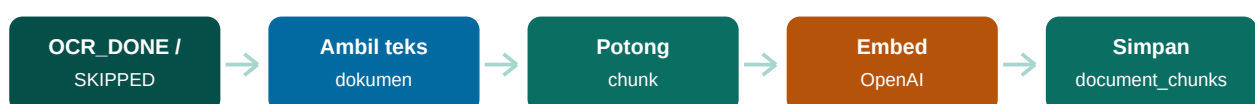


Gambar 13. Auto-scan dari favorit sampai batch ingest.

Parameter	Default / catatan
AUTO_SCAN_ENABLED (env)	false - jika true, memaksa worker on
auto_scan_enabled (DB)	Toggle Admin; digabung dengan env force
auto_scan_interval_minutes	Default 60
SCAN_MAX_FILES	Batas file per batch (hindari lonjakan beban)

7.4 Embedding backfill setelah OCR_DONE

Setelah SyncFile mencapai OCR_DONE (atau SKIPPED dengan ID dokumen), worker dapat membuat DocumentChunk: potongan teks OCR + embedding OpenAI. Backfill juga mengejar file lama yang belum punya chunk milik user tersebut.



Gambar 14. Embedding backfill untuk RAG / Ask AI / search semantik.

Tanpa OPENAI_API_KEY: embedding terbatas/nonaktif. Ask AI dan search semantik akan kurang optimal; keyword search tetap bergantung data OCR yang ada.

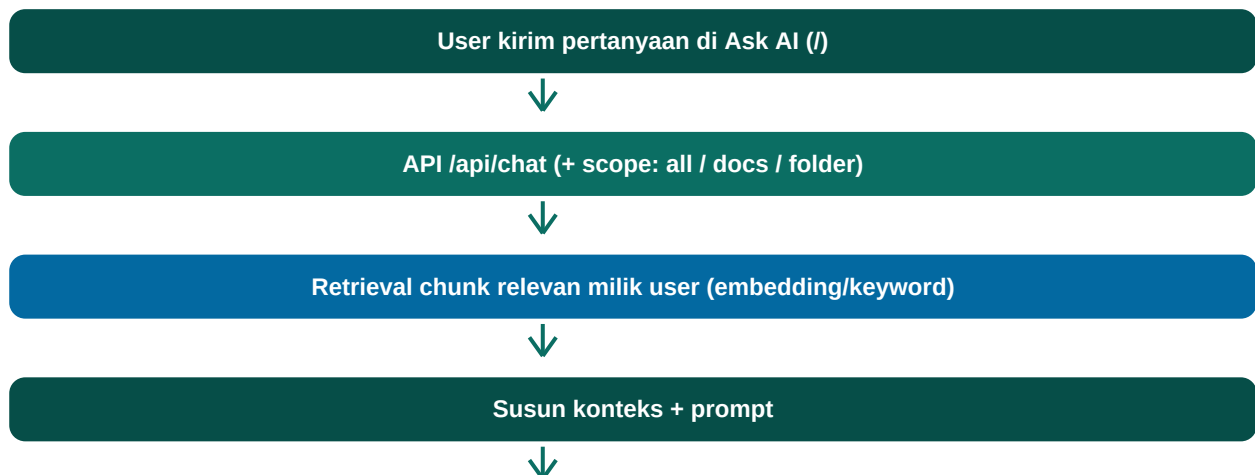
7.5 Ingest end-to-end

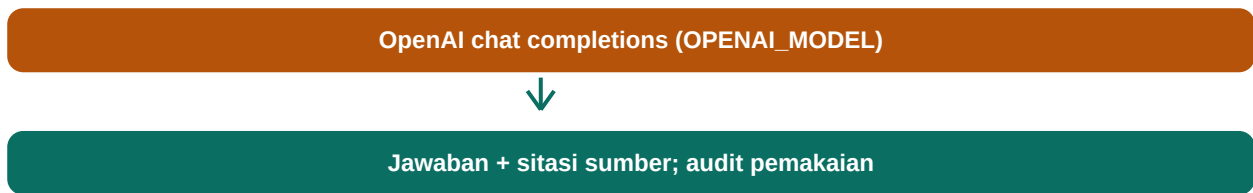
Dari klik user di Library sampai dokumen siap di Search/Ask AI:



Gambar 15. Pipeline ingest end-to-end.

7.6 Ask AI flow

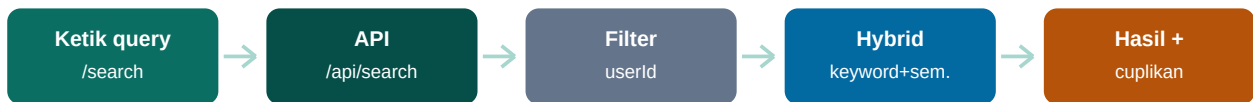




Gambar 16. Alur Ask AI (retrieval → model → jawaban).

Tanpa dokumen OCR_DONE + chunk, kualitas jawaban menurun tajam atau jawaban kosong.

7.7 Search flow



Gambar 17. Alur Search (isolasi per user).

- Hanya dokumen milik user yang sedang login (biasanya status OCR_DONE).
- Hybrid: keyword pada teks + semantik bila embedding tersedia.
- Ringkasan hasil search dapat memakai OPENAI_MODEL bila dikonfigurasi.
- Aktivitas dicatat di audit_logs (termasuk estimasi biaya AI bila ada).

F. Modul per halaman

8. Modul portal: Ask AI, Search, Library, Admin, Profil

Setiap modul di bawah mengikuti pola: tujuan → cara pakai singkat → API/komponen terkait → batasan.

8.1 Ask AI

Aspek	Isi
Route UI	/ (halaman utama chat)
Tujuan	Bertanya ke AI berdasarkan dokumen yang sudah di-OCR dan punya chunk.
Cara pakai	Login → buka Ask AI → pilih scope (semua / dokumen / folder) → kirim pertanyaan → baca jawaban + sitasi.
API / komponen	/api/chat, /api/chat/conversations, /api/chat/context, /api/chat/docs, AskWorkspace / ChatPanel
Batasan	Butuh OPENAI_API_KEY; kualitas bergantung OCR_DONE + embedding; tidak menggantikan baca dokumen hukum secara formal.



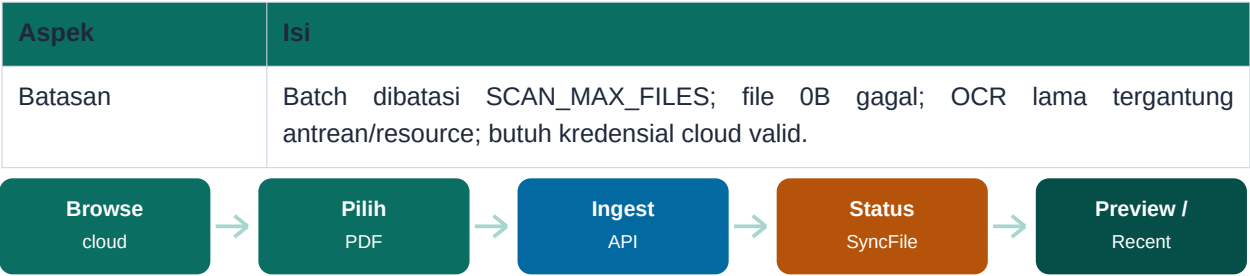
Gambar 18. Modul Ask AI (tinggi level).

8.2 Search

Aspek	Isi
Route UI	/search
Tujuan	Mencari isi dokumen terindeks (bukan hanya nama file).
Cara pakai	Ketik kata/frasa → lihat hasil & cuplikan → buka preview / lanjut Ask AI.
API / komponen	/api/search, SearchWorkspace; bisa terkait /api/cloud/recent
Batasan	File belum OCR_DONE tidak muncul; tanpa embedding, semantik lemah; ringkasan AI opsional bergantung key.

8.3 Library

Aspek	Isi
Route UI	/cloud, /cloud/recent
Tujuan	Browse cloud WebDAV, pilih PDF, fetch/OCR, lihat status, favorit, retry gagal, preview.
Cara pakai	Pastikan kredensial cloud di Profil → browse folder → pilih file/folder → Fetch/OCR → pantau status → recent / retry bila perlu.
API / komponen	/api/cloud/browse, ingest, folder-hint, recent, favorites, tree, retry, raw, count, search; CloudBrowser, FolderTreeSidebar, RecentScansList



Gambar 19. Modul Library.

8.4 Admin

Aspek	Isi
Route UI	/admin, /admin/users
Tujuan	Kelola pengguna, pantau audit/aktivitas AI & biaya, atur auto-scan.
Cara pakai	Login sebagai ADMIN → ringkasan/audit → kelola user (role, password, cloud) → toggle auto-scan bila perlu.
API / komponen	/api/admin, /api/admin/audit; AdminPanel, AdminUsersPanel, AdminAuditPanel
Batasan	Hanya role ADMIN; estimasi biaya bergantung data audit & pricing; auto-scan tetap dibatasi SCAN_MAX_FILES.

8.5 Profil

Aspek	Isi
Route UI	/profile
Tujuan	Kelola profil akun dan kredensial Cloud (disimpan terenkripsi).
Cara pakai	Isi URL cloud, username, password → simpan → uji browse di Library.
API / komponen	/api/profile; ProfileForm
Batasan	Tanpa kredensial valid Library tidak bisa browse; mengganti ENCRYPTION_KEY akan merusak data terenkripsi lama.

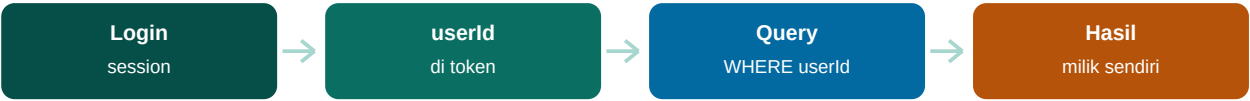
G. Keamanan & multi-user

9. Keamanan, isolasi data, role, audit, rate limit

9.1 Isolasi data per user (contoh konkret)

Setiap SyncFile, DocumentChunk, chat, dan favorit terikat userId. Query Search/Ask AI/Library selalu difilter ke user yang sedang login.

- User A meng-OCR folder /proyek-x → SyncFile & chunk milik A.
- User B login: Search tidak menampilkan dokumen A; Ask AI tidak mengambil chunk A.
- Meski paperless_document_id bisa sama secara isi (duplikat hash), kepemilikan logika tetap per user di DB app.
- Kredensial WebDAV A tidak dipakai untuk request milik B.



Gambar 20. Isolasi data per user.

9.2 Enkripsi kredensial cloud (ENCRYPTION_KEY)

Username dan password cloud disimpan terenkripsi di kolom encrypted_bappenas_*. Kunci berasal dari env **ENCRYPTION_KEY**. Worker/portal mendekripsi hanya saat perlu akses WebDAV.

PERINGATAN: jangan rotate / ganti ENCRYPTION_KEY setelah kredensial tersimpan. Data terenkripsi lama tidak bisa dibaca lagi, dan browse/ingest cloud akan gagal sampai kredensial diisi ulang.

9.3 Role ADMIN vs USER

Role	Bisa	Tidak (default)
USER	Library, Search, Ask AI, Profil, favorit, ingest milik sendiri	Halaman /admin, kelola user lain, lihat audit global
ADMIN	Semua kemampuan USER + Admin users/audit/auto-scan	Mengakses data cloud user lain tanpa kredensial mereka

9.4 Password hash bcrypt

Password login aplikasi disimpan sebagai password_hash (bcrypt), bukan plain text. Reset/ubah password lewat Admin atau alur yang disediakan; hash lama diganti hash baru.

9.5 Audit

Aksi penting (search, chat, admin, settings) ditulis ke audit_logs dengan meta JSON. Admin memakai /api/admin/audit untuk filter waktu, aktivitas, dan estimasi pemakaian AI.

9.6 Rate limit

Endpoint sensitif (misalnya browse cloud) dibatasi agar tidak mudah di-spam. Jika user mendapat kesalahan rate limit, tunggu sebentar lalu coba lagi.

H. OpenAI models & biaya

10. Model OpenAI, kapan dipanggil, dan audit biaya

DocSearch memakai OpenAI untuk dua kelas pekerjaan: **chat/completion** dan **embedding**. Model default dikonfigurasi lewat environment variable.

Pemakaian	Env	Default
Chat / Ask AI / ringkasan search	OPENAI_MODEL	gpt-4o-mini
Embedding (RAG & backfill worker)	OPENAI_EMBEDDING_MODEL	text-embedding-3-small

10.1 Kapan OpenAI dipanggil?



Gambar 21. Titik pemanggilan OpenAI.

- **Ask AI:** chat completions dengan konteks chunk yang di-retrieve.
- **Embedding untuk retrieval:** mengubah query/teks chunk agar pencarian semantik bekerja.
- **Embedding backfill di sync-worker:** setelah OCR_DONE, isi document_chunks.
- **Search summary:** ringkasan hasil bila fitur dikonfigurasi memakai model chat.

10.2 Audit pemakaian & estimasi biaya

Pemakaian dicatat lewat audit (meta berisi token/estimasi bila tersedia). Admin dapat melihat ringkasan hit AI dan estimasi biaya di panel audit. Angka bersifat perkiraan berdasarkan pricing yang dikonfigurasi di kode (mis. rate per 1M token untuk gpt-4o-mini), bukan invoice resmi OpenAI.

Tanpa OPENAI_API_KEY: Ask AI dan embedding terbatas atau tidak berfungsi. Pipeline OCR/Library tetap jalan, tetapi Search semantik & chat berbasis dokumen tidak optimal.

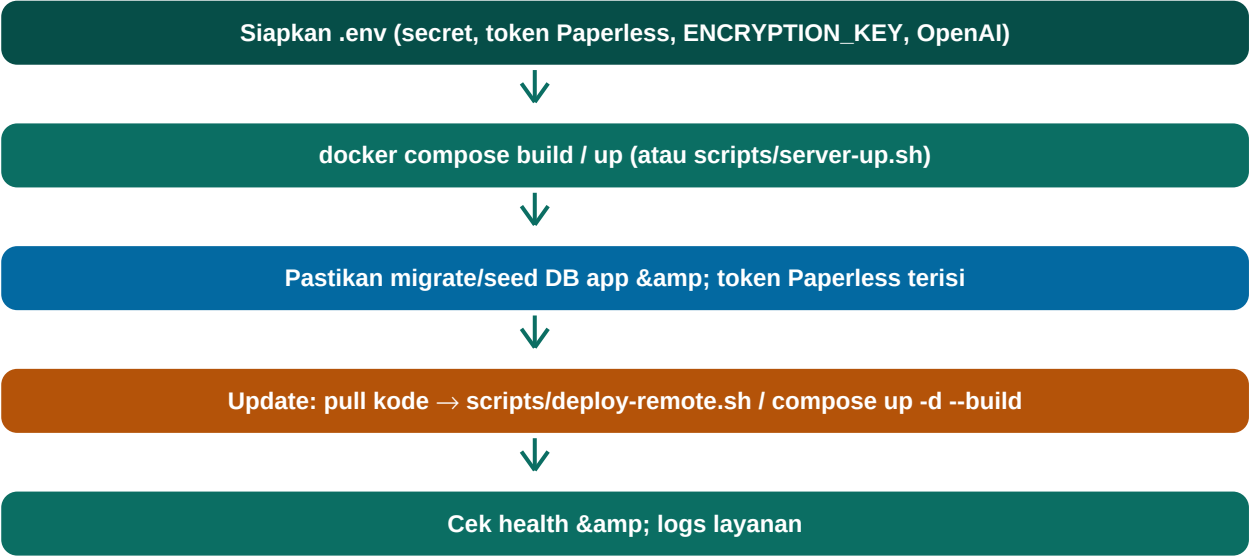
Kondisi	Dampak ke produk
Key valid + model default	Ask AI, embed, summary berjalan normal
Key hilang / invalid	Chat/embedding gagal; OCR & browse cloud tetap
Banyak ingest sekaligus	Lonjakan biaya embedding backfill - pantau SCAN_MAX_FILES
Banyak chat panjang	Lonjakan token chat - pantau audit Admin

I. Operasional

11. Deploy, backup, health, dan tuning 8 GB RAM

11.1 Deploy / update singkat

Layanan dijalankan dengan Docker Compose. Alur operasional tipikal:



Gambar 22. Konsep deploy / update.

- **docker compose:** orkestrasi postgres, redis, paperless, sync-worker, app.
- **scripts/server-up.sh:** membantu menyalakan stack di server.
- **scripts/deploy-remote.sh:** pola deploy/update jarak jauh (konsep: sync kode + rebuild).

11.2 Backup

Yang dibackup	Mengapa penting
Volume / data PostgreSQL	Berisi DB app (user, SyncFile, chat, audit) + DB paperless
Media & data Paperless	PDF terproses / hasil OCR tidak hanya di DB
File .env	Secret, token, ENCRYPTION_KEY, model OpenAI - simpan aman
(Opsional) volume consume	Transient; prioritas utama media+DB+env

Backup ENCRYPTION_KEY bersama data. Restore DB tanpa kunci yang sama = kredensial cloud tidak bisa didekripsi.

11.3 Health & monitoring ringan

- Endpoint **/api/health**: cek hidupnya portal (middleware mengizinkan tanpa auth penuh).
- **docker compose ps**: pastikan app, sync-worker, paperless, postgres, redis Up.
- **docker compose logs** (app / sync-worker / paperless): unduh gagal, OCR timeout, antrean.
- Di UI: pantau status FAILED / OCR_PENDING di Library; di Admin pantau audit AI.

11.4 Tuning untuk sekitar 8 GB RAM

Paperless biasanya konsumen memori terbesar. Di mesin 8 GB, jaga beban OCR dan batch scan:

Parameter	Saran praktis	Alasan
PAPERLESS_TASK_WORKERS / threads	1 / 1	Cegah OOM saat OCR
PAPERLESS_WEBSERVER_WORKERS	1	Cukup untuk API internal
SCAN_MAX_FILES	20 (atau lebih kecil)	Batasi lonjakan unduh+OCR+embed
OCR_PENDING_TIMEOUT_MINUTES	180 (default)	Beri waktu OCR; sesuaikan jika antre panjang
Interval reconcile / auto-scan	Jangan terlalu agresif	Kurangi CPU idle thrash
PAPERLESS_MEM_LIMIT	mis. 1536m	Batasi kontainer OCR

J. Lampiran

12. Glosarium, FAQ, checklist go-live

12.1 Glosarium

Istilah	Arti sederhana
OCR	Mengenali teks dari gambar/PDF scan
WebDAV	Protokol akses file di cloud (list/download)
Ingest / consume	Memasukkan file ke pipeline OCR Paperless
Consume folder	Folder pantauan Paperless untuk file masuk
Embedding	Representasi angka dari teks untuk pencarian semantik
RAG	Jawab AI dengan mengambil cuplikan dokumen relevan dulu
BullMQ	Sistem antrai pekerjaan di Redis
SyncFile	Catatan status satu file cloud di database app
ScanJob	Batch pekerjaan discovery/unduh/OCR
Reconcile	Mencocokkan SyncFile ke dokumen Paperless (checksum → ID)
content_hash	Checksum isi file untuk duplikat & pencocokan
paperless_document_id	ID dokumen di Paperless setelah OCR
SKIPPED	Status: isi sudah ada / tidak perlu OCR ulang
FAILED	Status: gagal (OB, timeout, unduh, dll.)
ENCRYPTION_KEY	Kunci enkripsi kredensial cloud at rest
OPENAI_MODEL	Model chat (default gpt-4o-mini)
OPENAI_EMBEDDING_MODEL	Model embedding (default text-embedding-3-small)
Paperless-ngx	Mesin open-source OCR & arsip dokumen
AUTO_SCAN	Ingest otomatis folder favorit sesuai interval
SCAN_MAX_FILES	Batas jumlah file per batch scan/ingest

12.2 FAQ

Pertanyaan	Jawaban singkat
Kenapa file tidak muncul di Search?	Belum OCR_DONE (masih pending/gagal), atau bukan milik user login. Cek status di Library; pastikan reconcile selesai.
Kenapa OCR lama?	Antre Paperless, PDF besar/scan berat, worker=1 di RAM terbatas, atau banyak batch bersamaan. Pantau OCR_PENDING & logs.
Apa itu file 0B?	File kosong di cloud. Tidak dikirim OCR; status FAILED. Perbaiki sumber di cloud lalu retry.
Kredensial cloud error?	Isi ulang di Profil. Pastikan URL/username/password benar. Jangan ganti ENCRYPTION_KEY setelah data tersimpan.
Ask AI kosong / lemah?	Belum ada chunk/embedding, OPENAI_API_KEY hilang, atau scope dokumen kosong. Selesaikan OCR dulu, cek key, perluas scope.
Apa bedanya SKIPPED dan OCR_DONE?	Keduanya bisa dipakai bila ada document ID. SKIPPED biasanya duplikat isi (hash sama) tanpa OCR ulang.
Auto-scan tidak jalan?	Cek toggle Admin, AUTO_SCAN_ENABLED, interval, dan apakah ada favorit. Batch tetap dibatasi SCAN_MAX_FILES.

12.3 Checklist go-live

- .env lengkap: NEXTAUTH_SECRET, ENCRYPTION_KEY, Postgres, PAPERLESS_SECRET_KEY, admin Paperless.
- PAPERLESS_API_TOKEN diisi dan dicoba (preview/reconcile berhasil).
- OPENAI_API_KEY diisi jika Ask AI / embedding dibutuhkan; model default dicek.
- Docker Compose: semua layanan Up; /api/health OK.
- User ADMIN dibuat; user uji punya kredensial cloud valid di Profil.
- Uji alur: browse → ingest 1 PDF kecil → tunggu OCR_DONE → Search → Ask AI.
- Uji file 0B / retry / FAILED terlihat benar di UI.
- Backup: rencana volume Postgres + media Paperless + salinan .env aman.
- Tuning 8 GB: workers Paperless=1, SCAN_MAX_FILES terkendali.
- Auto-scan: biarkan off dulu di produksi sampai pantauan beban stabil.
- Admin audit: pastikan aktivitas chat/search tercatat.
- Dokumentasikan kontak operasional (siapa restart compose, siapa pegang secret).

Penutup. DocSearch memakai Paperless-ngx sebagai fondasi OCR yang terbukti, lalu membungkusnya dengan portal multi-user, worker sync cloud, database aplikasi, embedding, dan Ask AI. Hasilnya bukan sekadar penyimpanan PDF, melainkan sistem agar dokumen organisasi bisa diambil, dibaca mesin, dicari, dan ditanyakan secara terkendali dengan status yang jelas dan jejak audit.